ITU-MiniTwit

A Report on Design, Operation, and Evolution

Group C - youCanCUs

Alexander Rossau <ross@itu.dk>
Alexander Frederiksen <alefr@itu.dk>
David Nicholas Nielsen <davn@itu.dk>
Anders Georg Frølich Hansen <ageh@itu.dk>
Phillip Nikolai Rasmussen <phir@itu.dk>

BSc DevOps, Software Evolution and Software Maintenance
IT University of Copenhagen - Spring 2026

19/05/2026

Contents

1 Introduction

Author: Alexander Rossau

This report documents the design, operation, and evolution of *ITU-MiniTwit*, a Twitter-like microblogging platform built as part of the DevOps, Software Evolution and Software Maintenance course at ITU Copenhagen, Spring 2026. Users can register, post short messages ("cheeps"), follow other authors, and comment on posts. The system was re-implemented from a legacy Flask application into a C#/ASP.NET stack and deployed on cloud infrastructure using Docker Swarm. The source code is hosted at GitHub. Operational metrics are available in Grafana and logs are aggregated through Loki and visualized in Grafana.

2 System's Perspective

2.1 Architecture and Design

Author: Alexander Rossau

Allocation viewpoint Figure 1 shows the production deployment. The system runs as a Docker Swarm stack on a DigitalOcean droplet. This can be deployed with a few Terraform commands, using the included Terraform files in the `infrastructure` directory.

Caddy acts as a reverse proxy with IP-hash load balancing, forwarding HTTP traffic to the `chirp-web` application container. The application connects to a PostgreSQL 17 database over the internal Docker overlay network. Observability is co-located: Prometheus scrapes application metrics (exposed via `/metrics`) and a `postgres-exporter` sidecar. Promtail ships container logs to Loki and Grafana queries both Prometheus and Loki for dashboards and alerting. Shepherd polls the GitHub Container Registry and performs rolling updates when a new image tagged `latest` is published. We have used this rolling-update pattern since the start of the project to enable zero-downtime deployments, with manual redeploys and database migrations being the exceptions where downtime did occur.

On our main deployment used in this course, we use Tailscale Funnel to expose the service to the internet on ports 80 and 443. This allows access to the service from anywhere, without exposing the server's public IP address directly. The tradeoff with this is that all requests take a latency penalty, as they are routed through the Tailscale network, which is not meant for high-throughput production traffic. In a real production environment, we would use a custom domain and configure it in Caddy to handle the traffic.

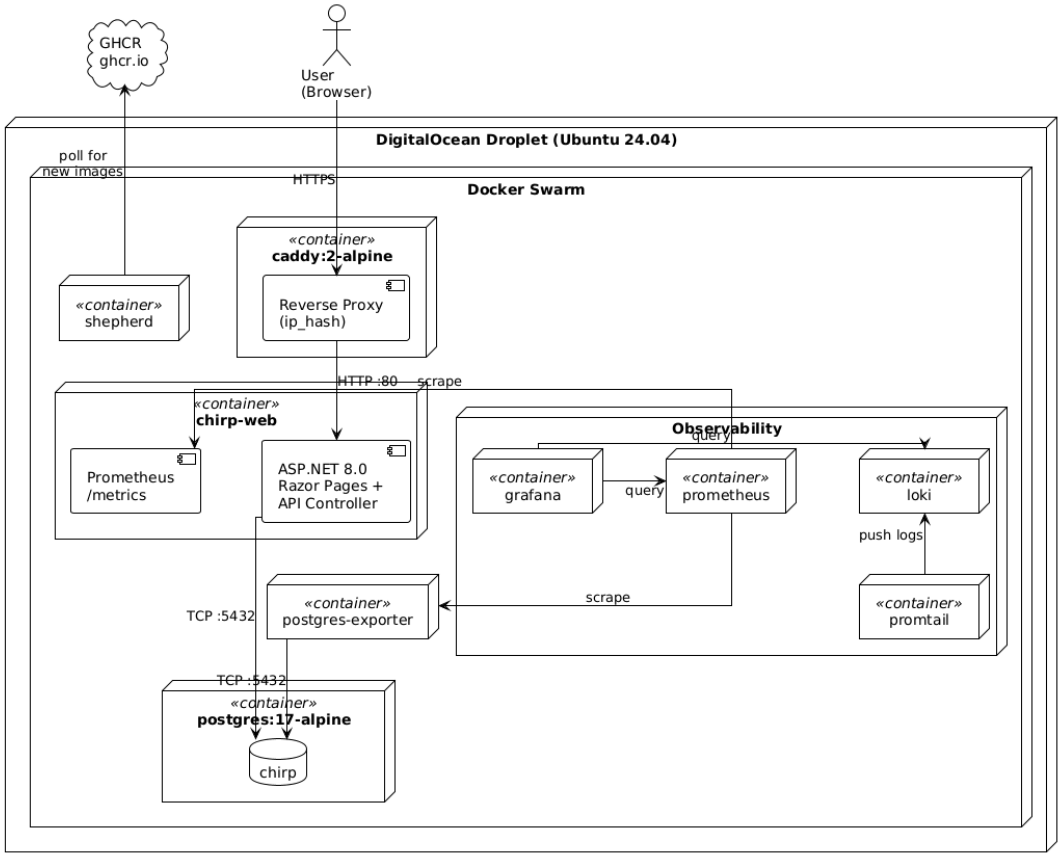

Figure 1: Allocation viewpoint - UML deployment diagram of the production system.

Module viewpoint. Figure 2 illustrates the package decomposition. The codebase follows an onion architecture across four .NET projects. `Chirp.Core` defines the domain model (`Author`, `Cheep`, `Follow`, `Comment`) and DTOs with no outward dependencies. `Chirp.Repositories` provides Entity Framework Core access to PostgreSQL through `ChirpDBContext` and depends only on `Core`. `Chirp.Services` contains business logic (`CheepService`) and depends on repository interfaces. `Chirp.Razor` is the outermost layer, hosting ASP.NET Razor Pages for the web UI and an API controller that implements simulator endpoints (`/msgs`, `/fllws`, `/register`). Prometheus counters (`ChirpMetrics`) are recorded in this layer. This separation allows the CI pipeline to run integration tests against a temporary database instance, without the web layer.

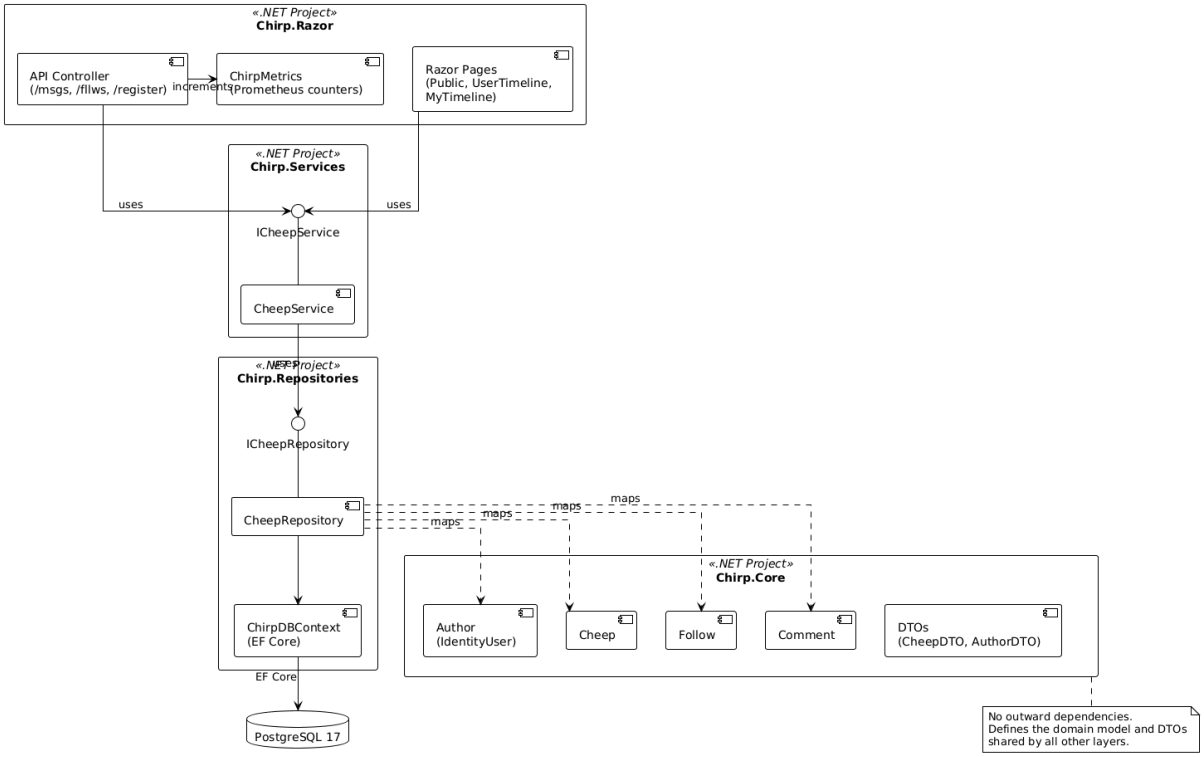

Figure 2: Module viewpoint - component/package decomposition.

Component & Connector viewpoint. Figure 3 traces the "post a cheep" flow through the system. A simulator `POST /msgs/{username}` request is received by Caddy, forwarded to the API controller, which validates the authorization header, resolves the author through `CheepService`, and delegates persistence to `CheepRepository`. After the cheep is inserted into PostgreSQL, the controller increments the `chirp_cheeps_created_total` Prometheus counter and returns `204 No Content`.

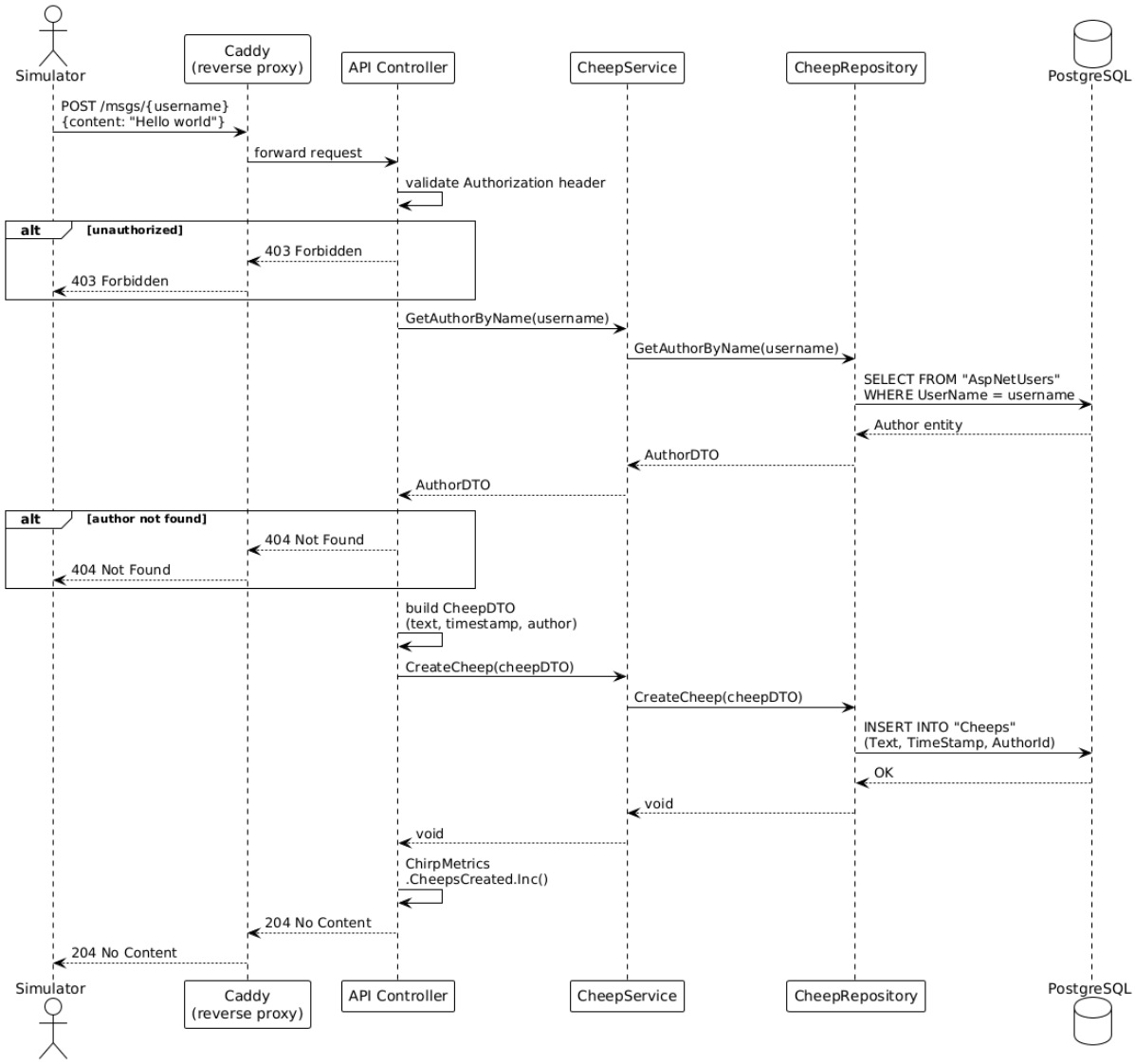

Figure 3: Component & Connector viewpoint - sequence diagram for posting a cheep.

2.2 Dependencies

Author: Alexander F

Layer	Tool / Technology	Purpose
Runtime	ASP.NET Core (.NET 8)	HTTP server, request handling
ORM	Entity Framework Core	Database access
Data	PostgreSQL (Docker container, accessed via Npgsql and EF Core)	Primary persistence
Containerization	Docker	Service isolation & deployment
Infra	Docker Swarm + Terraform (DigitalOcean)	Hosting & orchestration
CI/CD	GitHub Actions	Build, test, deploy
Observability (Monitoring)	Prometheus	Metrics collection
Observability (Logging)	Loki + Promtail	Log aggregation
Observability (Visualization)	Grafana	Dashboard visualization

Table 1: Key dependencies, by layer.

Our MiniTwit system is built in C# and runs with ASP.NET Core via .NET as our web framework. The data is stored in PostgreSQL database running in a Docker container using the Npgsql provider. Our program is containerized using Docker.

The system is deployed using Docker Swarm, and the infrastructure is from using Terraform.

For observability, we are using Prometheus to collect metrics from the program, while Loki and Promtail handle the log aggregation and Grafana is used to visualize both the metrics and the logs through its dashboards.

2.3 Current State

Author: Alexander F

Our system uses GitHub Actions for continuous integration and automated validation. The CI pipeline performs static analysis, automated builds, database-backed testing, and browser-based integration testing with every commit.

The project contains four different kinds of automated tests: unit tests, integration tests, UI tests, and end-to-end tests. These tests are done in the CI pipeline using `make test`.

Static analysis and security scanning are done using Semgrep with rulesets targeting C#, Dockerfiles, and the OWASP Top 10. The CI workflow also provisions a PostgreSQL container during testing to support the integration tests against a database environment. The end-to-end testing includes a Playwright browser automation test for the web UI.

The system as it is right now contains 34 build warnings like 'nullable-reference warnings', 'logging-analysis warnings', 'unused-variable warnings' and 'test-analyzer warnings'. In addition to 1 known package warning (`NU1903` related to the `Microsoft.Build 17.8.3`). These warnings mostly concern nullable-reference handling and package dependency issues. Even though these warnings are present, our system builds fine and runs successfully.

Our repository has 3 GitHub Actions workflows (`ci.yml`, `devskim.yml`, and `release-docker.yml`) supporting the automated testing, the security scanning, and also the deployment processes.

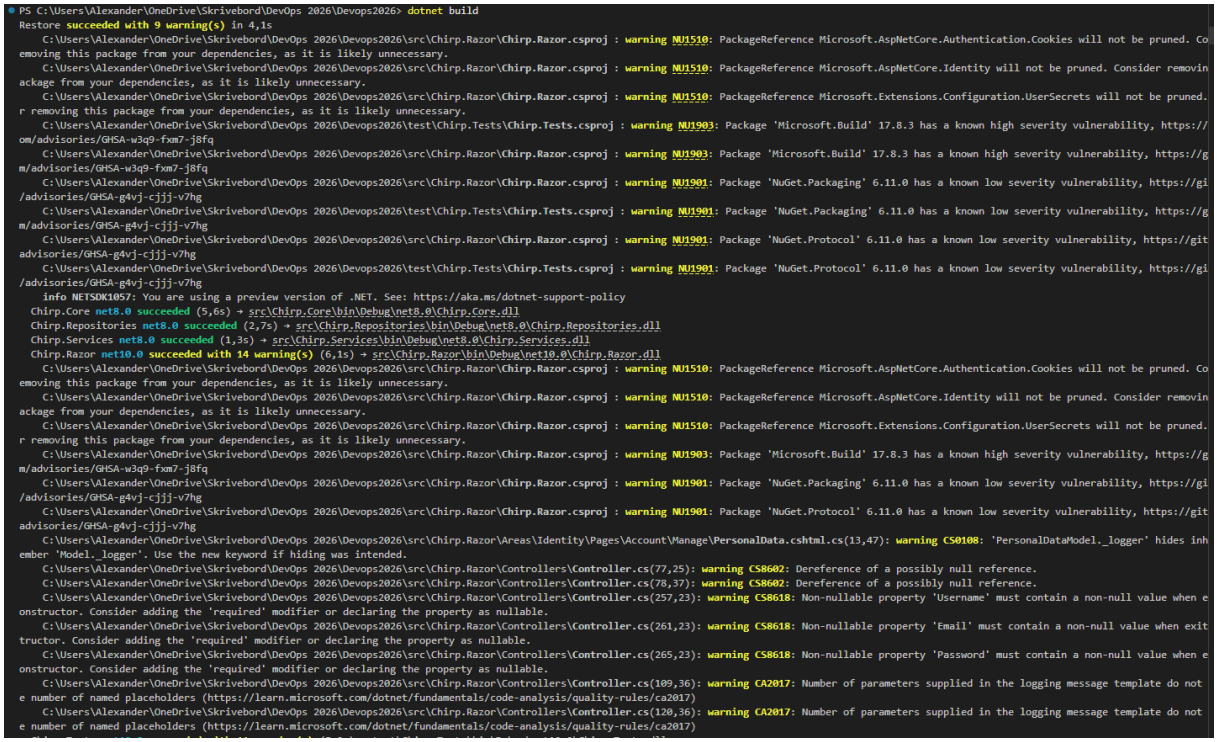

Figure 4: Warnings part 1

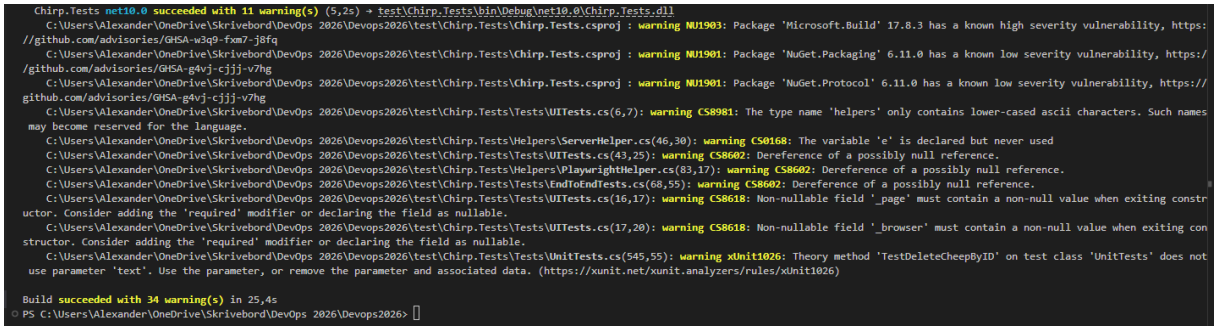

Figure 5: Warnings part 2

3 Process Perspective

3.1 CI/CD Pipeline

Author: David Nicholas Nielsen

The CI/CD pipeline is implemented using the following tools:

- We use Git and GitHub for version control and code hosting.
- We use GitHub Actions for automated testing and building the application.
- We use Docker for containerization, GitHub Container Registry for container image hosting, and Docker Swarm for orchestration and deployment.
- We use Terraform for infrastructure-as-code (IaC), provisioning our DigitalOcean droplet and Docker Swarm cluster.

Regarding GitHub Actions, we have set up the following workflows:

1. **CI**: runs on every push and pull request, executing tests, as well as static analysis with Semgrep. It enforces quality gates by failing if tests do not pass or if Semgrep finds critical issues.
2. **DevSkim Security Scan**: runs on every push to the `main` branch, scanning for security issues with DevSkim and failing if any are found.
3. **Build and Push Docker Image on Release**: runs whenever a new release is published, building a new Docker image, pushing it to GitHub Container Registry.

The pipeline is illustrated in the figure below.

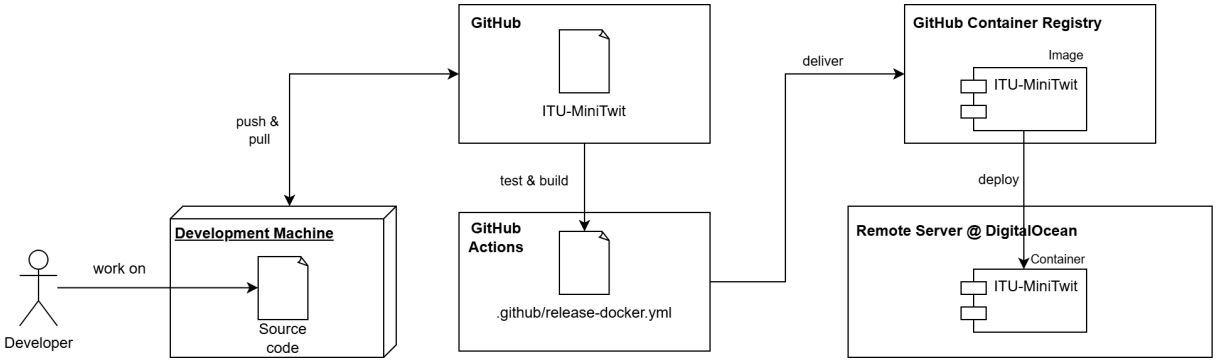

Figure 6: CI/CD Pipeline. This diagram shows the flow from local code changes to deployment. Note that other containers are also running on the server, such as those used for the database and the monitoring stack.

3.2 Deployment and Release

Author: David Nicholas Nielsen

Docker Swarm is configured to automatically update to use the newest available release of ITU-MiniTwit. We use a tool called *Shepherd* that periodically checks for new releases. If a new release is found, it performs a rolling update of the stack, meaning that the new version is deployed to one replica at a time. In case of a failed deployment, the stack will automatically roll back to the previous version.

3.3 Availability and Scaling

Author: David Nicholas Nielsen

Currently, the system always creates exactly one replica of each service, and if a replica fails, the container is restarted. To increase availability, we could scale horizontally by increasing the number of replicas and rely on Caddy's existing LB. The number of replicas could be set to a higher number, or it could be adjusted dynamically based on the load. This would allow the system to handle more traffic and provide tolerance in case of failures. Of course, in a real-world scenario, simply increasing the number of replicas may not be enough to ensure high availability, as we would need to consider other possible bottlenecks, such as the single-replica database.

3.4 Monitoring

Author: Anders Georg Frølich Hansen

When managing a website, availability is key. Monitoring facilitates availability and is an important aid in noticing and diagnosing a problem with a web application.

In this project we use Grafana for visualization and Prometheus to collect and provide the data that is displayed. Prometheus scrapes the data from a frontend and a backend source.

The frontend data comes from an exposed endpoint on the website, "/metrics". This endpoint primarily provides business relevant metrics, such as how many cheeps or comments are created by users. Prometheus also scrapes the backend data through postgres-exporter. This data can be split up into reactive- and proactive monitoring. We use reactive monitoring to see whether the database is active. Furthermore, we also use proactive monitoring to be able to identify problems in advance. Examples include monitoring the size of the database and also how high the cache hit rate is. For an example of proactive monitoring, when we monitor the database size we could set an alarm that notifies on 10% available disk space remaining.

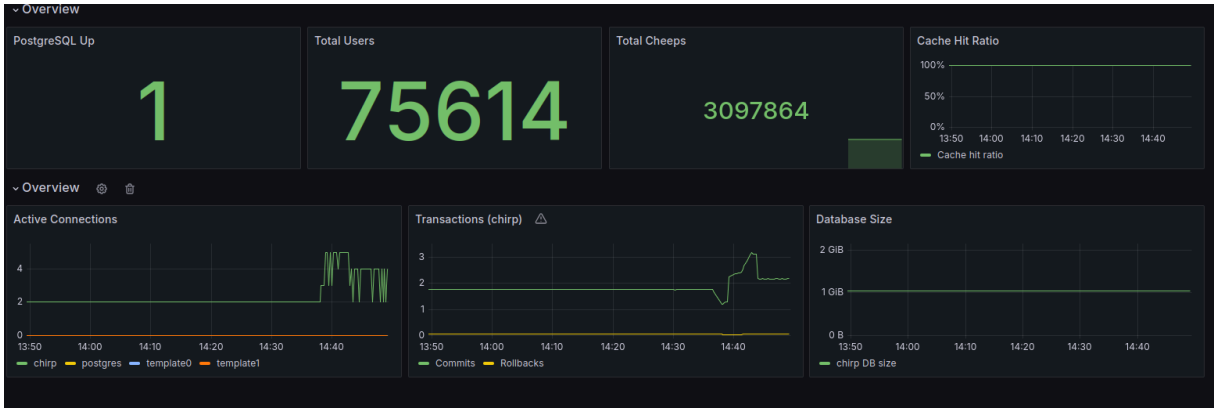

Figure 7: Monitoring overview of postgres database

3.5 Logging

Author: Anders Georg Frølich Hansen

Just as monitoring is key for availability, logging is key for diagnosing a problem. For this project we use the built-in logging functionality in the .NET library. Promtail collects these logs from the docker containers together with OS logs. They are then all sent to Loki, which is responsible for aggregating and storing them. At last, the logs are displayed chronologically in Grafana.

The logs are sent as structured JSON objects and include information that can help us debug/ recall what has happened, such as: containerid, POST/GET, endpoint, response time, IP-address, and a description of what was logged.

We log many different things. Below are some grouped examples:

Security events
- A user fails/succeeds to register
- A user fails/succeeds to login

Business-critical operations
- A user's timeline is accessed
- A user follows/unfollows another user
- A user creates/deletes a cheep

Errors
- Exceptions

System events
- System logs

Performance can also be tracked through response time, if this is too slow, something could be wrong and a request is instead logged as a warning.

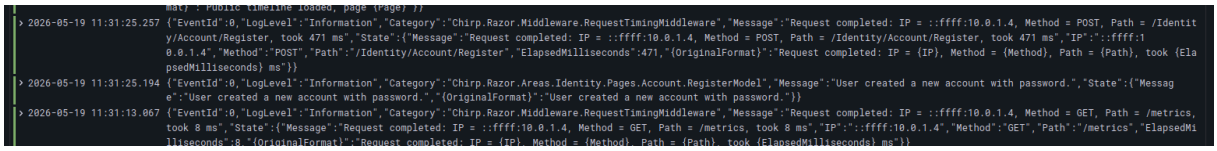

Figure 8: Example of a log of a registered user

3.6 Security Hardening

Author: Anders Georg Frølich Hansen

Security is important, especially since our application contains sensitive information such as passwords. The following sections describe how we try to implement the defense-in-depth model.

TLS
All communication between clients and the server is secured using HTTPS, ensuring that data is encrypted in transit.

Hashing
User passwords are never stored in plain text. Instead, they are stored using salted hashing.

Secret scanning and vulnerability checks
As part of the CI/CD pipeline, we automatically scan the codebase for accidentally committed secrets such as passwords. This is done using DevSkim and Semgrep with rules based on the OWASP Top 10. These tools help identify common vulnerabilities such as SQL injection.

Network
All communication between clients and the server happens through a reverse proxy. This adds an additional security layer and the opportunity to filter various malicious requests before they ever reach the server. For example, a flood from a single IP can be rate-limited at the reverse proxy.

We keep as few ports open as possible. In Figure 9 our firewall rules can be seen.

```
root@srv1039100:~# ufw status
Status: active

To                         Action      From
--                         ------      ----
Anywhere on tailscale0     ALLOW       Anywhere
22                         ALLOW       100.64.0.0/10
22/tcp                     ALLOW       Anywhere                   # SSH from anywhere
2377/tcp                   ALLOW       Anywhere
7946/tcp                   ALLOW       Anywhere
7946/udp                   ALLOW       Anywhere
4789/udp                   ALLOW       Anywhere
Anywhere (v6) on tailscale0 ALLOW       Anywhere (v6)
22/tcp (v6)                ALLOW       Anywhere (v6)              # SSH from anywhere
2377/tcp (v6)              ALLOW       Anywhere (v6)
7946/tcp (v6)              ALLOW       Anywhere (v6)
7946/udp (v6)              ALLOW       Anywhere (v6)
4789/udp (v6)              ALLOW       Anywhere (v6)
```

Figure 9: Overview of firewall rules using ufw

Least privileges
Every container is run with the least possible privileges. As seen in Figure 10, most containers are not run as the root user.

```
devops-group@srv1039100:~/Devops2026$ bash containerUsers.sh
Container: 9ebdbe68d812  Image: ghcr.io/anders0106/devops2026:latest@sha256:335053f05bd8908c97a02d15df7627bc7e1aa5fbca854c6cfd40eb495b929a59
uid=1000 gid=1000 groups=1000

Container: 7dde948c604a  Image: postgres:17-alpine@sha256:979c4379dd698aba0b890599a6104e082035f98ef31d9b9291ec22f2b13059ca
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)

Container: 62d042852af3  Image: grafana/loki:3.6.0@sha256:6a705de65df88aa0d90a44779606b0042722c86637335a73858f65f9fe9f9557
OCI runtime exec failed: exec failed: unable to start container process: exec: "id": executable file not found in $PATH

Container: c864ce35c658  Image: caddy:2-alpine@sha256:86deaf5e3d3408a6ccec08fbb79989783dd26e206ae10bcf78a801dc8c9ab794
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)

Container: 4cc97f38d679  Image: grafana/promtail:3.6.0@sha256:2aafa34b3d5fba888c51081d3a22c234906ffd3cafc5def11c581549b297d449
uid=0(root) gid=0(root) groups=0(root)

Container: 3587a4f0225d  Image: grafana/grafana:11.2.0@sha256:408afb9726de5122b00a2576763a8a57a3c86d5b0eff5305bc994ceb3eb96c3f
uid=472(grafana) gid=472 groups=472

Container: c40130234760  Image: prometheuscommunity/postgres-exporter:v0.15.0@sha256:386b12d19eab2a37d7cd8ca8b4c7491cc7a830d9581f49af6c98a393da9605e6
uid=65534(nobody) gid=65534(nobody) groups=65534(nobody)

Container: f6fc02e20797  Image: prom/prometheus:v2.52.0@sha256:5c435642ca4d8427ca26f4901c1111402300470903788 0cd7860d5b7176aa731
uid=65534(nobody) gid=65534(nobody) groups=65534(nobody)

Container: 0761d33de022  Image: containrrr/shepherd:latest@sha256:b117c2394832e088932d5e1eebb8df6c1924f47d0fef31788cb310c0fe3bf7db
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
```

Figure 10: Overview of what user each container from Docker Swarm is run as

4 Reflection Perspective

Author: Phillip Nikolai Rasmussen

We had some issues figuring out why a lot of follow and cheep API requests were failing. It ended up being a combination of many small issues not considered when the functions were first created, such as case-sensitive usernames and whitespace handling. This can be seen in commits `3428504`, `4b511f6`, and `f05129f`. The issue still persisted, so we added logging in commit `5d61e5d` to highlight exactly where things were going wrong.

Another big issue was the migration from SQLite to PostgreSQL (commit `315f361`). SQLite had been handling certain values case-insensitively, which PostgreSQL does not, so we needed a conversion script to fix the existing data (commit `be71171`). On top of that, a lot of tests broke, leading to a string of quick fixes from commits `65de226` to `970e37c`, not fully resolved until `8c816e2` two weeks later. This caused minor downtime for our application, while we were working on migrating the database.

During the maintenance phase, we went through a round of security hardening after realizing the system had some gaps. In a series of commits we added Caddy as a reverse proxy (`9adb05c`), bound all service ports to localhost (`356062b`), removed root as the running user from our containers (`4113645`), and added Semgrep for static security analysis (`ee46c2e`). Going through this as a dedicated step made it clear how many small attack surfaces a running system can accumulate without much notice.

The DevOps style of our work was focused on improving the CI/CD flow constantly throughout development. The focus was on fixing issues and making sure that if they reappear, we know right away, along with trying to automate as many things as possible, which made the codebase much easier to work with as the project progressed.

5 Use of Generative AI

Author: Phillip Nikolai Rasmussen

During the project we used Claude (Sonnet 4.6 and Opus 4.6) and GPT-5.4 as generative AI assistants. The two providers were used interchangeably, choosing whichever gave cleaner output for a given task.

Learning new tooling Terraform and Docker Swarm were new to the team. Rather than reading documentation from scratch, we used AI to get targeted explanations and working examples we could adapt to our setup. This significantly reduced the time needed to become productive with each tool.

Boilerplate generation Integrating Prometheus, Loki, and Grafana involves a lot of repetitive configuration. AI generated base templates that we then reviewed and adjusted, avoiding the most mechanical parts of the work.

Test fixes Approximately 15 existing tests broke during the initial migration. AI diagnosed the failures and proposed fixes quickly. Not a hard task, but one that would have consumed more time without assistance.

Logging instrumentation Once we agreed on a logging convention, AI applied it consistently across the codebase, turning a tedious but low-value task into a matter of minutes.

AI improved our velocity noticeably. The clearest benefit was lowering the barrier to unfamiliar tooling, because getting a working starting point is often the hardest step. The main drawback was overconfident output, particularly with Terraform provider-specific syntax, which occasionally introduced subtle errors we only caught through testing. This taught us to treat AI output as a draft to verify rather than a finished answer, and to always cross-reference official documentation for infrastructure-critical configuration.